

23. Verilator

What is Verilator?

- Verilator is an open source compiler for SystemVerilog (and Verilog)
 - It translates SystemVerilog into C++
 - The C++ code can then be executed to simulate the design
- Can include C++ models and SystemC (a library for modelling hardware in C++)
- Can link to other languages through its own API (Application Programming Interface)
 - and through the SystemVerilog DPI and VPI (200 pages of the SystemVerilog LRM)

How to get Verilator

- See <https://veripool.org/verilator/>
- Runs under Windows, Linux, MacOS
 - Precompiled versions are available
- Can be compiled from source (github)
- Needs several prerequisites
 - Including a C++ compiler, even for the prebuilt version
 - Verilator translates to C++, so that code still needs to be compiled
- Also install GTKWave (<https://gtkwave.sourceforge.net/>) for waveform display

Verilator Limitations

- Verilator does not implement all of the IEEE 1800 standard
 - It's not a full SystemVerilog simulator (unlike ModelSim/QuestaSim, Vivado simulator, VCS, etc.)
 - Supports synthesisable RTL code
 - Need to use blocking and nonblocking assignments correctly
 - Cycle-based, not gate delay-based
 - 2-state logic (no 'X' or 'Z')
 - Support for classes and other verification constructs is still limited

Running Verilator

- It's easiest to use a Makefile
 - Follow certain conventions:
 - 1 module per file
 - Module name and filename should match (decoder should be in decoder.sv)
 - Testbench name should match top-level module (test_decoder used as testbench for decoder)
 - Leave a blank line at the end of each file
 - Include \$finish in the testbench
 - Use \$dumpfile to save simulation results
 - But also use \$display to print info

Example

- Combinational Decoder

```
module decoder (input logic [1:0] a,  
                output logic [3:0] y);
```

```
  always_comb  
    case (a)  
      0 : y = 1;  
      1 : y = 2;  
      2 : y = 4;  
      3 : y = 8;  
    endcase
```

```
endmodule
```

Example

- Testbench

```
module test_decoder; //No i/o
    logic [1:0] a;
    logic [3:0] y;

    decoder d0 (.*) ;
//only works if internal
//signals have the same names
initial
    begin
        $dumpfile("waveform.vcd");
        $dumpvars(0,test_decoder);
```

```
        #10ns a = 0; // initial
// value of a is not defined
        #10ns a = 1;
        #10ns a = 2;
        #10ns a = 3;
        #10ns $finish;
//Verilator needs $finish
    end

    always @(a)
        $display("a = %d,
                y = %d", a, y);

endmodule //blank line at end
```

Makefile (extracts)

```
MODULE=test_decoder
```

Change MODULE to match testbench name

```
waves: waveform.vcd
    gtkwave waveform.vcd
```

```
waveform.vcd: ./obj_dir/V$(MODULE)
    @./obj_dir/V$(MODULE) +verilator+rand+reset+2
```

```
./obj_dir/V$(MODULE): .stamp.verilate
    make -C obj_dir -f V$(MODULE).mk V$(MODULE)
```

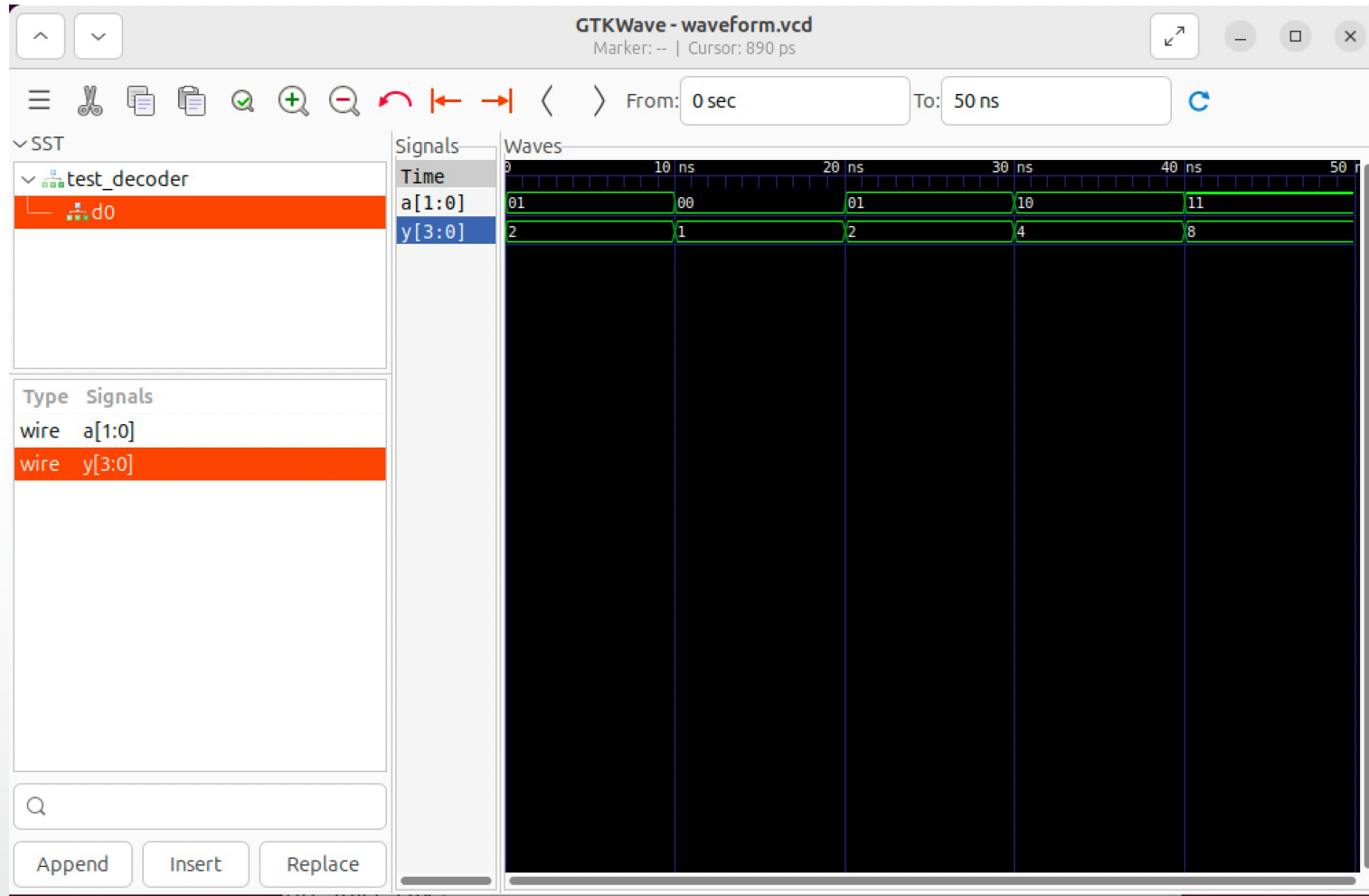
```
.stamp.verilate: $(MODULE).sv
    verilator -Wall -Wno-ENUMVALUE -Wno-UNUSED SIGNAL --trace \
        --x-assign unique --x-initial unique --binary \
        -cc $(MODULE).sv --exe
    @touch .stamp.verilate
```

Verilator will find other files automatically

Executing Makefile

- Type:
`make waves`
- This will "verilate" the SystemVerilog file to create
`Vtest_decoder.cpp` in `obj_dir` folder
- Then compile `Vtest_decoder.cpp` to create executable
`Vtest_decoder`
- Then run `Vtest_decoder`, which creates `waveform.vcd`
file
- Then open `waveform.vcd` in GTKWave

GTKWave



Output from simulation

Lots of messages about compilation

```
a = 1, y = 2
a = 0, y = 1
a = 1, y = 2
a = 2, y = 4
a = 3, y = 8
- test_decoder.sv:16: Verilog $finish
- Simulation Report: Verilator 5.045 devel
- Verilator: $finish at 50ns; walltime 0.000 s; speed 840.916 us/s
- Verilator: cpu 0.000 s on 1 threads; allocated 6 MB
```

Makefile

- Clean up with
`make clean`
- It is possible to list all the files that make up a project in the Makefile, to show dependencies
- For small projects, it's probably easiest to type `make clean` and `make` again after any changes

Test RISC-V CPU

Test RISC-V CPU

- Compile C code (add.c) with

```
riscv32-unknown-elf-gcc -c -march=rv32i -o  
add.o add.c
```

- Compile assembly (add.s) with

```
riscv32-unknown-elf-as -c -march=rv32i -o  
add.o add.s
```

- Convert object file to hex text

```
riscv32-unknown-elf-objcopy -O verilog add.o  
add.txt
```

Disassembled Object File

0:fd010113	addi sp, sp, -48
4:02812623	sw s0, 44(sp)
8:03010413	addi s0, sp, 48
c:fca42e23	sw a0, -36(s0)
10:fc42c23	sw a1, -40(s0)
14:fdc42783	lw a5, -36(s0)
18:00e78793	addi a5, a5, 14
1c:fef42623	sw a5, -20(s0)
20:fec42783	lw a5, -20(s0)
24:00078513	mv a0, a5
28:02c12403	lw s0, 44(sp)
2c:03010113	addi sp, sp, 48
30:00008067	ret

Hex file

```
@00000000
```

```
13 01 01 FD 23 26 81 02 13 04 01 03 23 2E A4 FC  
23 2C B4 FC 83 27 C4 FD 93 87 E7 00 23 26 F4 FE  
83 27 C4 FE 13 85 07 00 03 24 C1 02 13 01 01 03  
67 80 00 00
```

- Notice that each instruction has its byte order reversed
 - 13 is at address 0,
 - 01 is at address 1,
 - 01 is at address 2
 - FD is at address 3

Read contents of text file into memory

```
module test_readmemh;  
  
logic [7:0] mem [60];  
logic [31:0] instruction;  
  
initial  
  begin  
    $readmemh("add.txt", mem);  
    for (int pc = 0; pc < 60; pc+=4)  
      begin  
        instruction = {mem[pc+3], mem[pc+2], mem[pc+1], mem[pc]};  
        $display("%h %h", pc, instruction);  
      end  
    $finish;  
  end  
endmodule
```

Test_readmemh output

```
000000000 fd010113
000000004 02812623
000000008 03010413
00000000c fca42e23
000000010 fcb42c23
000000014 fdc42783
000000018 00e78793
00000001c fef42623
000000020 fec42783
000000024 00078513
000000028 02c12403
00000002c 03010113
000000030 00008067
```

- We have reconstructed the 32-bit instructions inside a SystemVerilog simulation
- This can be used to verify a RISC-V core
- Make the instructions as simple, or as complex, as you wish